OffBloom — Complete Application Guide

Oct 29, 2026 · @Charly

1. OffBloom on one page

OffBloom is a digital wellbeing coach that, instead of blocking apps, analyses how you use your phone and proposes a weekly plan with offline activities tailored to you. Its motto is **"substitute, don't forbid"**: scrolling fills a gap (rest, boredom, stress), and if you only take it away, the gap fills up with the same habit again.

Who it is for. The primary persona is a professional aged 25 to 40 who scrolls late at night and wants to win back sleep and quality time without giving up their phone. The design leaves room for students struggling with focus and for knowledge workers with digital burnout.

What makes it different.

- **It is agentic.** A graph of three agents (Analyst, Planner and Curator) reasons over your profile and usage instead of returning a fixed list.
- **It is grounded.** Every proposed activity comes from a curated corpus via RAG and carries a citation of its source; the coach does not make up advice.
- **It is safe by design.** A safety gate stops the flow when it detects risk signals and points the person to professional help instead of planning.
- **It is privacy-first.** It can run 100% locally on an open model (Ollama); no data leaves your machine.

What a user can do today. Create an account (email and password, or Google), complete an onboarding with their goals and interests, check in with their phone usage, receive a plan with cited activities, tick off the ones they complete, browse their plan history, edit their profile and delete their account with all its data.

Stack in one line. Python 3.12 + FastAPI + LangGraph + ChromaDB on the backend, SQLAlchemy + Alembic + SQLite for persistence, Next.js 16 + React 19 + Tailwind v4 on the frontend, with 216 tests, model evals with DeepEval and CI on GitHub Actions.

Code. github.com/carlosalba4/offbloom. Every non-obvious decision is recorded as an ADR in `docs/decisions/`. The app's interface is in Spanish, which is why the screenshots in this guide are too.

2. The problem and the philosophy

Apps that only block fail because they treat the symptom: the phone was covering a real need, and taking it away without a substitute leaves a gap that pushes people into relapse. OffBloom chooses to accompany people and offer something better to do in that gap.

Substitute, don't forbid. The heart of the product is the Curator, which proposes offline alternatives based on the person's available time, energy, place and moment. It is not a generic list: with 15 minutes, low energy and at home, the proposal is different from an hour free outdoors.

Grounded advice, not invented advice. Every activity is retrieved from a curated corpus on digital wellbeing and shown with the citation of its source. The user knows where each proposal comes from and the model does not improvise.

Privacy as a capability of the architecture. The LLM layer is provider-agnostic: the same app runs on a local model (Ollama with `qwen3:8b`) or a hosted one (OpenAI, Google or Anthropic) by changing one environment variable. Anyone who wants full privacy can run everything on their own machine. A hosted public demo would use a cloud provider, and that is stated plainly: privacy is an option the architecture allows, not a promise about any particular deployment.

Safety is blocking. Wellbeing is a sensitive domain. When there are risk signals (persistent distress, usage that seriously interferes with sleep, work or relationships, a feeling of being unable to stop) the coach does not plan: it responds with empathy and points to professional help, with no clinical advice and no invented phone numbers. A dedicated eval measures this behaviour against a threshold, and failing it is treated as blocking.

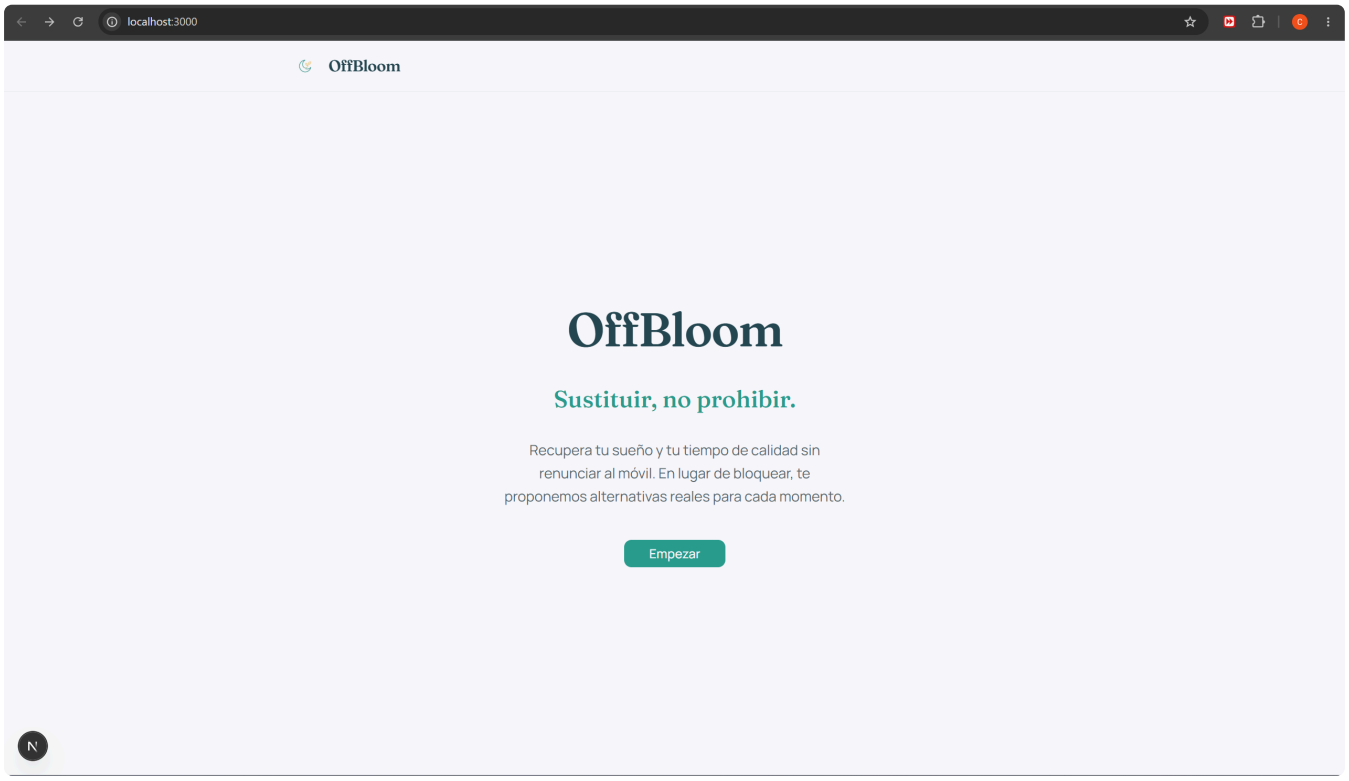
Out of scope, on purpose. There is no native mobile app and no real reading of the operating system's screen time: usage is self-reported in the check-in. There is no gamification or social features either. The focus is on making the core (the coach) solid.

3. App walkthrough

Typical use is five steps: create an account, describe your goals once, check in with your phone usage, receive a plan and tick off what you do. These are the screens in that order. The interface is in Spanish; each caption explains what the screen shows.

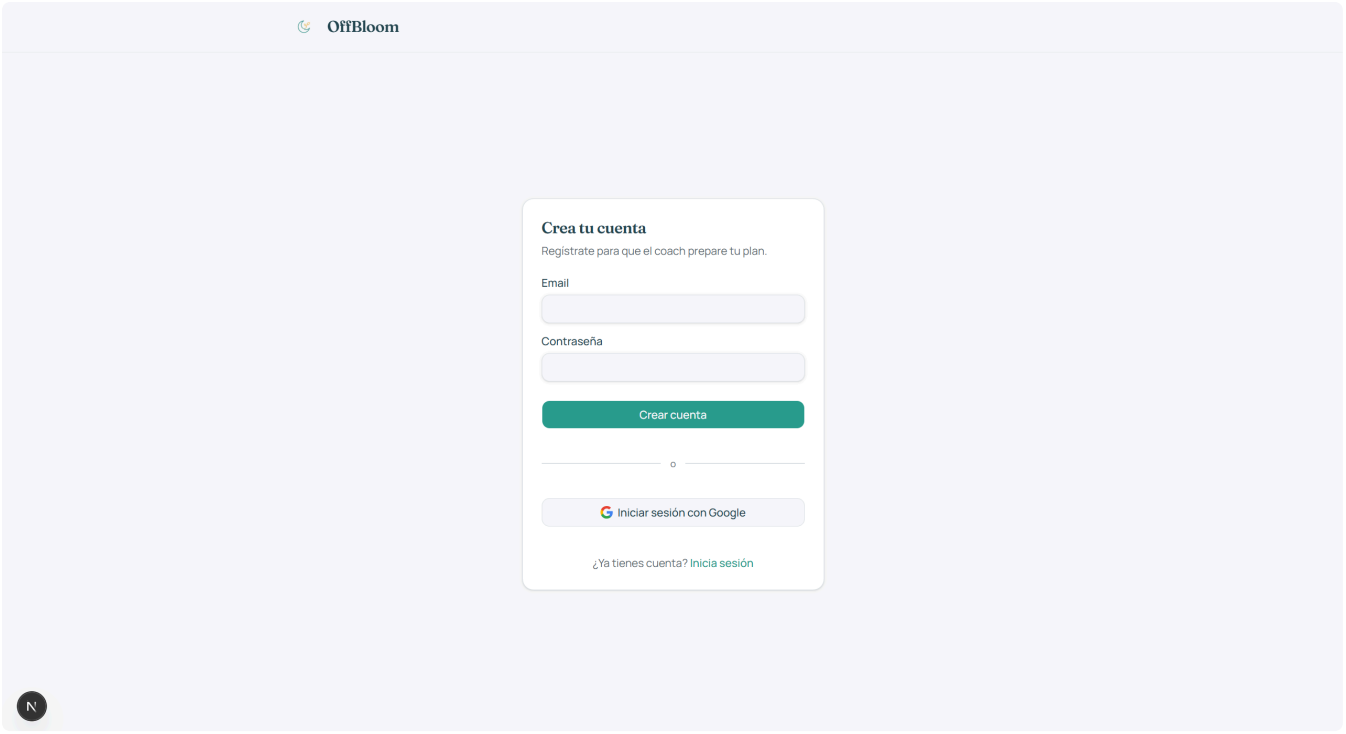
Landing page

The home page sums up the proposition in one line, "Sustituir, no prohibir" (substitute, don't forbid), and a single **Empezar** (get started) button. If you already have a session, the button takes you straight into the app.



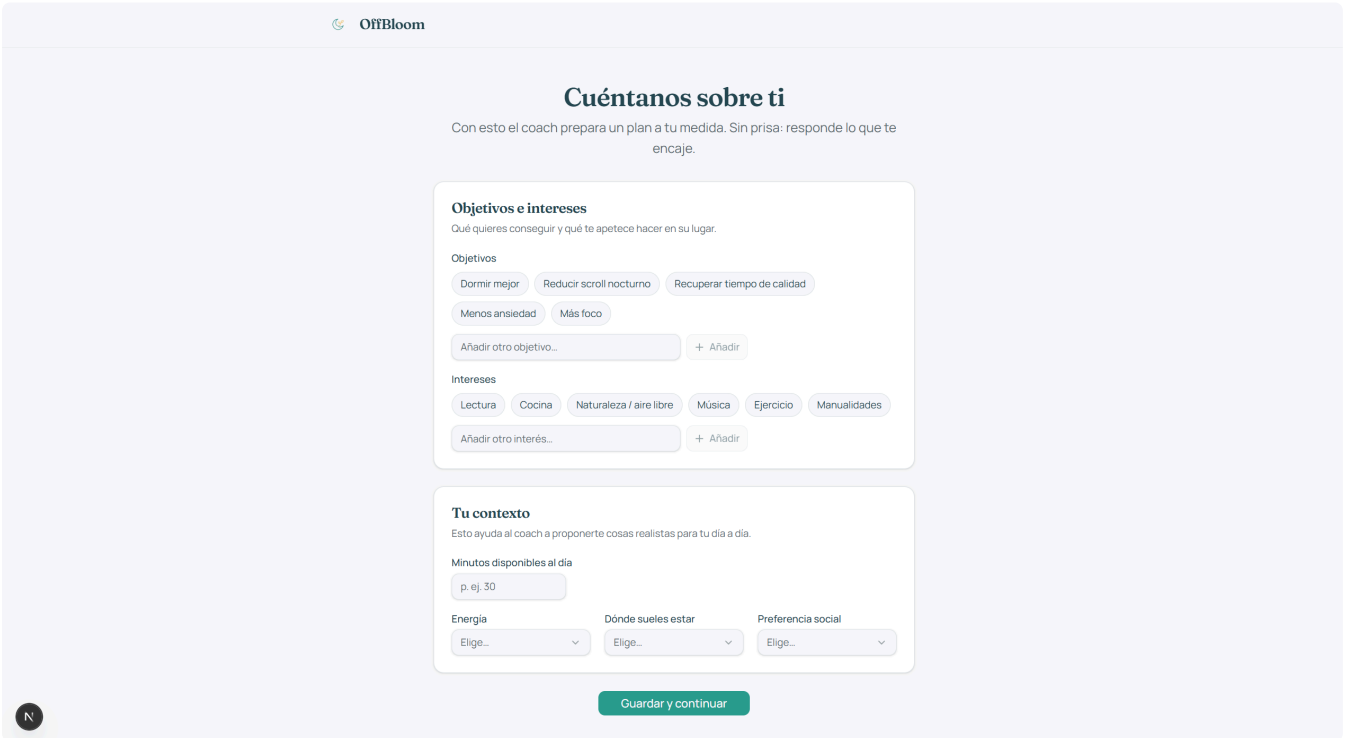
Sign up and sign in

You can create an account with email and password or with **Iniciar sesión con Google** (sign in with Google). Both routes lead to the same account and the same session: if you signed up with email and later sign in with Google using that same verified email, OffBloom recognises you. The sign-in screen looks the same.



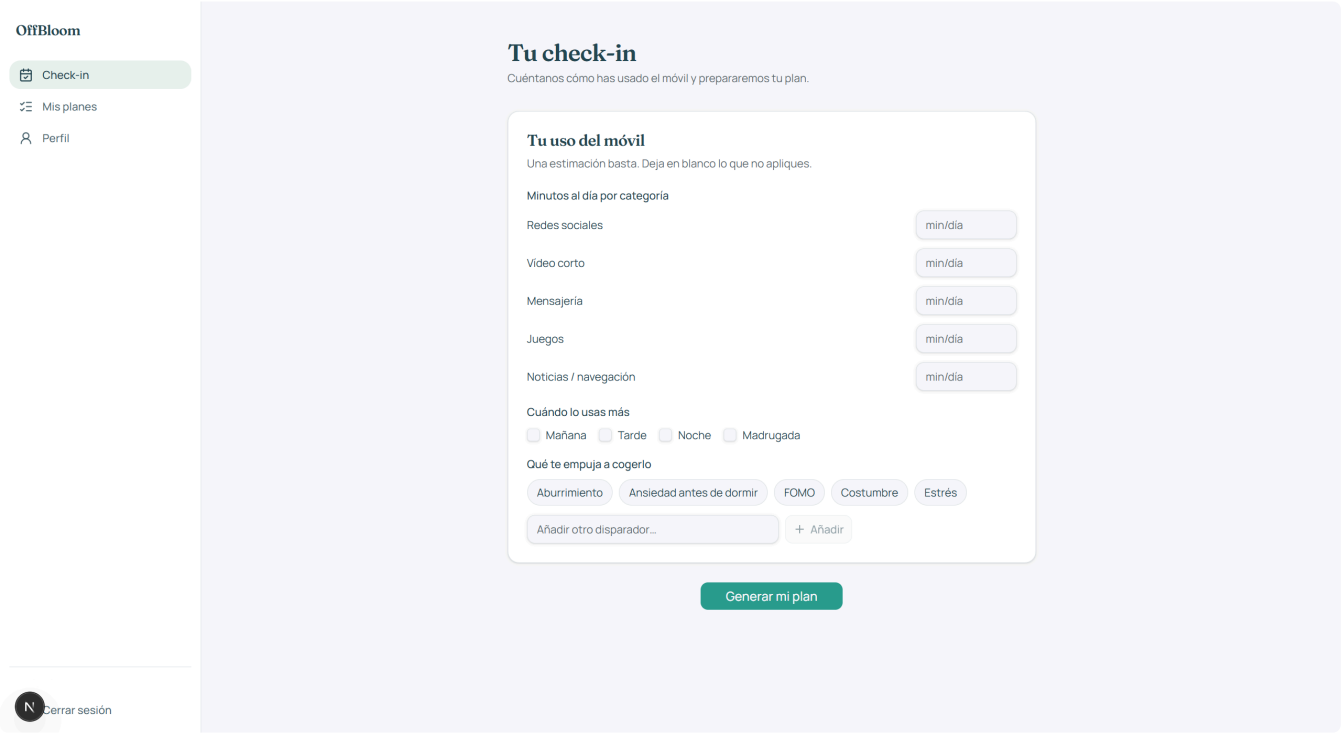
Onboarding: tell us about yourself

The first time, the app asks for what the coach needs to personalise: **goals** (sleep better, cut down late-night scrolling, more focus…), **interests** (reading, music, exercise…) and your **context**: free minutes per day, energy level, where you usually are and whether you prefer activities alone or with others. You can add your own goals and interests. Until you complete it, the app does not let you move to any other screen.



Check-in: your phone usage

Whenever you want a new plan, you do a quick check-in: an estimate of minutes per day by category (social media, short video, messaging, games, news), the times of day you use your phone most and what pushes you to pick it up (boredom, anxiety before bed, FOMO, habit, stress). No precision is needed: an estimate is enough. When you press **Generar mi plan** (generate my plan), the coach analyses all of this together with your profile.



Your plan

The plan is the heart of the app and has four parts:

1. **Focus for the week:** one sentence that sums up what to concentrate on.
2. **Intentions:** concrete, achievable commitments.
3. **Substitutions:** each habit struck through with its alternative next to it. This is "substitute, don't forbid" made visible.
4. **Activities for you:** three offline activities chosen for your context, each with why it fits you, which habit it replaces, its duration, the energy it needs and where to do it.

Each activity has a checkbox to mark it as done: it gets struck through, dimmed, and the "X de 3 completadas" (X of 3 completed) counter updates instantly. At the bottom are the **sources** from the corpus that ground the plan.

If the coach detects in the check-in that the distress goes beyond a habit, it does not generate a plan: it shows a careful message encouraging the person to lean on a professional or someone they trust.

TU FOCO DE ESTA SEMANA

Reducción del scroll nocturno para mejorar el sueño y la calidad del descanso

TUS INTENCIONES

- Activar el modo 'no molestar' 1 hora antes de dormir para evitar interrupciones
- Revisar mensajes solo en bloques definidos (ej: 15 minutos) en lugar de de forma continua
- Sustituir el último uso de pantallas por una actividad tranquila 30-60 minutos antes de acostarse
- Crear una playlist de música relajante para reemplazar el consumo de videos cortos
- Incorporar 10 minutos de estiramientos o respiración profunda como alternativa a la mensajería nocturna

SUSTITUCIONES

~~Scroll en redes sociales y videos cortos (noche)~~ →
Reproductor de música relajante con temporizador de 20 minutos

~~Mensajería nocturna para reducir aburrimiento~~ →
Grabar un mensaje de voz para enviar más tarde o hacer una llamada breve

~~Uso de pantallas antes de dormir~~ →
Preparar la rutina del día siguiente mientras se hace una ducha tibia

ACTIVIDADES PARA TI

2 de 3 completadas

☐ **Llamar a un familiar**

Encaja en el contexto de uso nocturno y cumple con la restricción de lugar (casa). Reemplaza el disparador de aburrimiento con una actividad social que requiere menos energía y tiempo (15 min).

Sustituye a: soledad, aburrimiento

15 min · Energía: Media · En cualquier sitio

☑ ~~**Repasar fotos impresas o recuerdos**~~

Se ejecuta en casa (lugar), requiere baja energía y cumple con el tiempo disponible (15 min). Reemplaza el disparador de scroll por una experiencia limitada y significativa.

Sustituye a: scroll

15 min · Energía: Baja · En casa

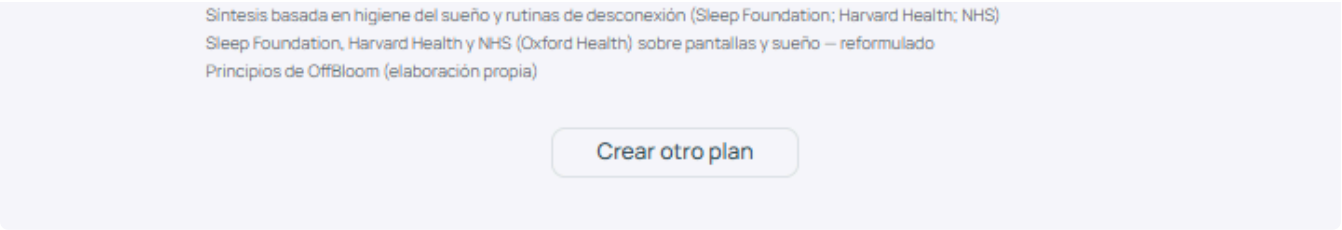
☑ ~~**Planificar un plan que te ilusione**~~

Requiere solo 10 min, baja energía y se adapta al momento de la tarde. Crea un plan ilusionante que distrae del impulso nocturno de scroll.

10 min · Energía: Baja · En cualquier sitio

FUENTES

Síntesis basada en diseño persuasivo y economía de la atención (Fogg Behavior Model; Center for Humane Technology)



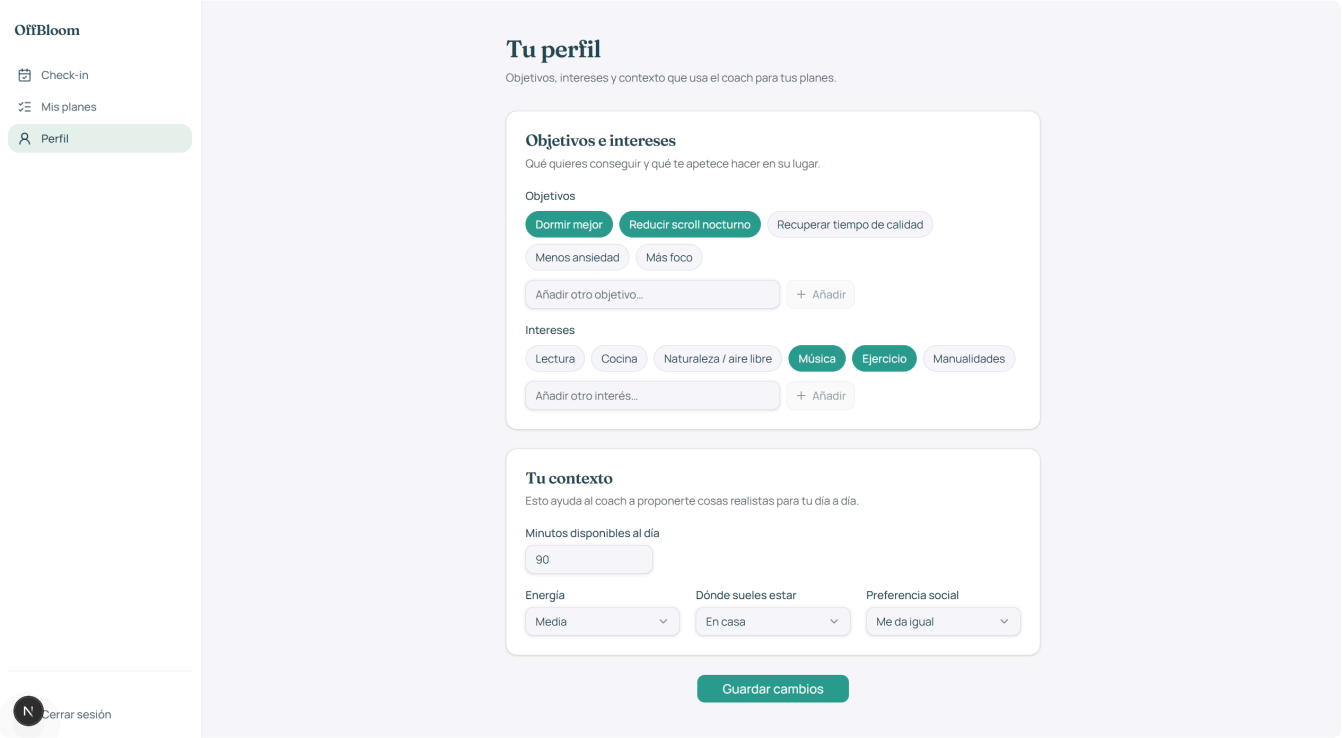
My plans

The history keeps all your plans, newest first, with their focus, their type (plan or referral) and the date. Opening one shows the full plan with the progress you had made, and you can keep ticking off activities.



Profile

Here you edit the same data as in the onboarding when your goals or routine change; the next plan takes them into account. You can also sign out from the sidebar.



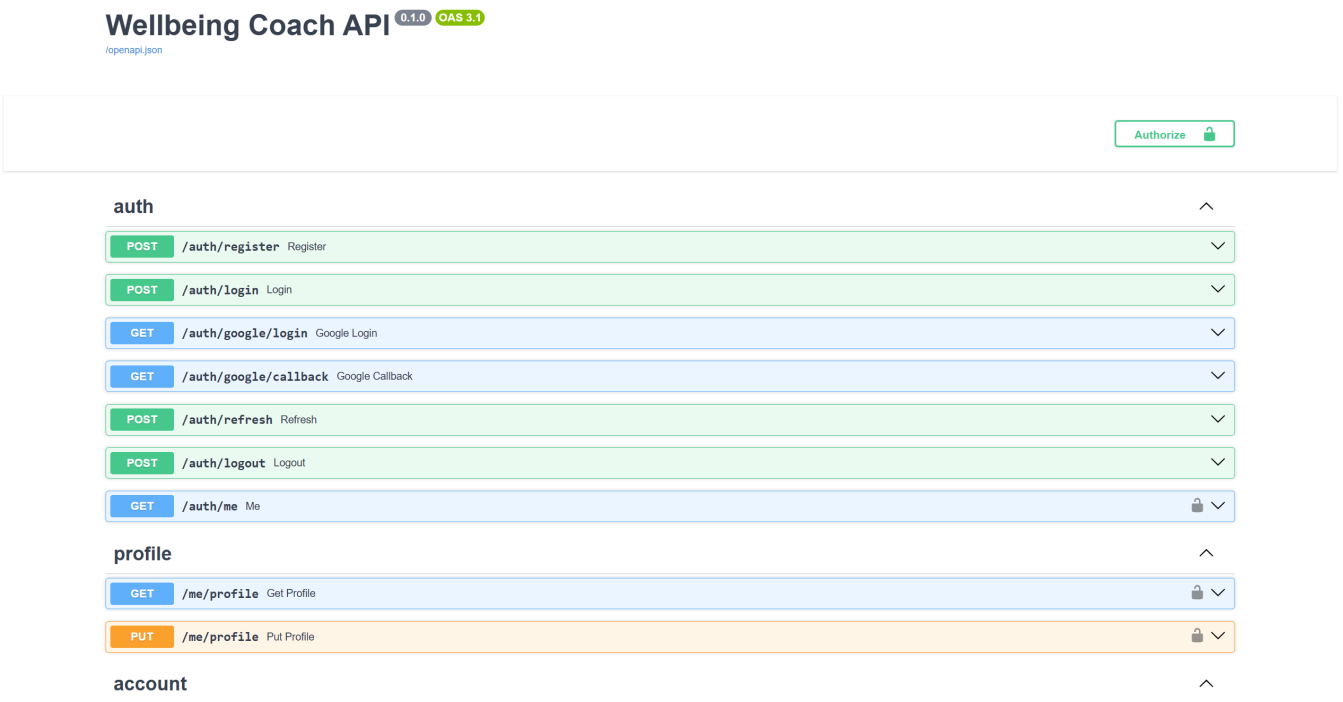
On mobile

The interface is responsive: on narrow screens the sidebar becomes a top navigation bar.


OffBloom
Check-in
Mis planes
Perfil
Cerrar sesión
Mis planes
El historial de tus planes semanales y derivaciones.
Reducir el scroll nocturno y reemplazar la estimulación pasiva por actividades que fortalezcan el bienestar físico y emocional
Plan
29 de octubre de 2026

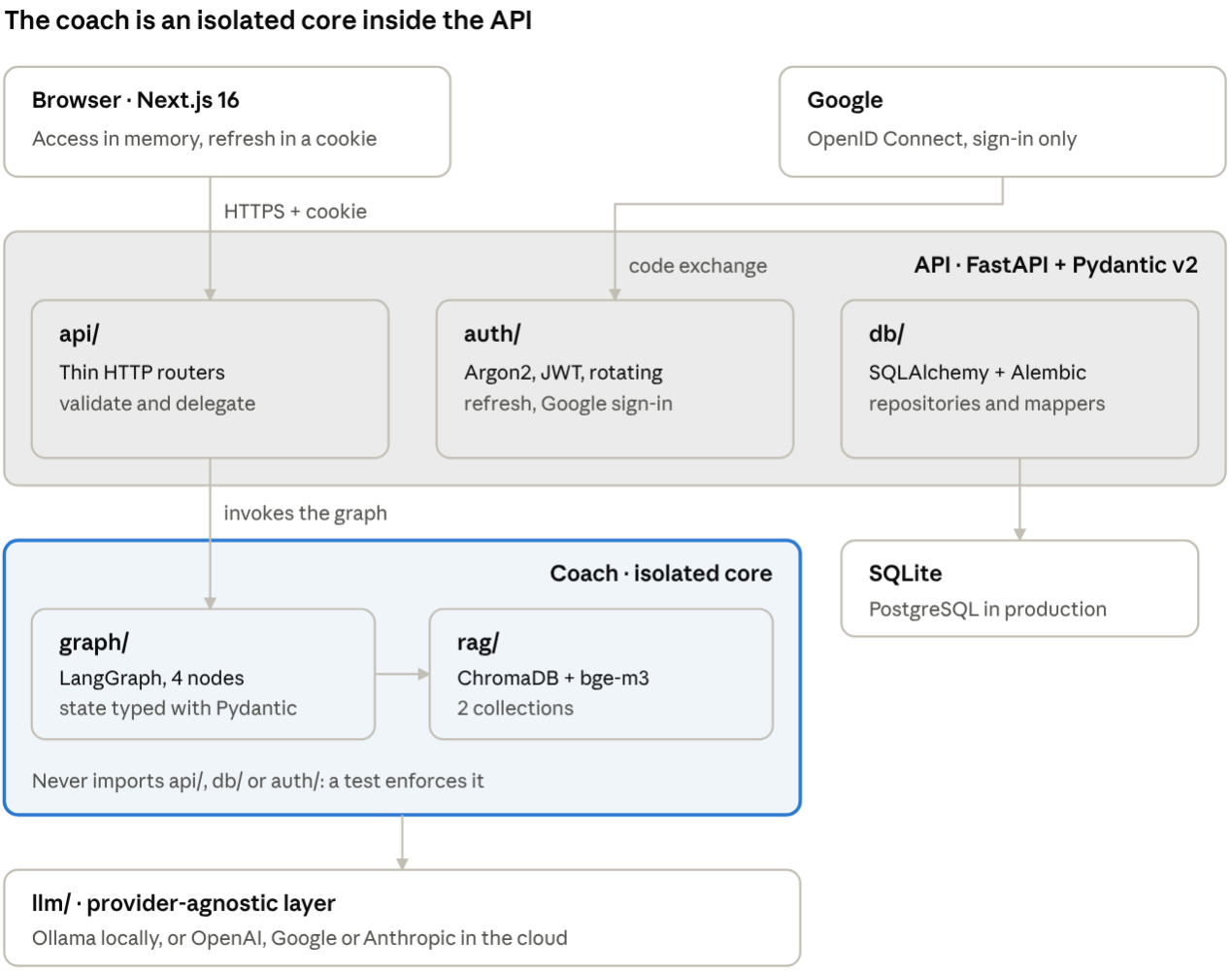
The API under the hood

For anyone who wants to see the backend, FastAPI generates interactive documentation at `/docs` (Swagger), where every endpoint can be tried out. The full reference is in the appendix.



4. Overall architecture

OffBloom has two pieces deployed separately: a **Next.js frontend** running in the browser and a **FastAPI API** holding all the logic. The API is organised in layers with one strict rule: the coach is an isolated core that knows nothing about users, databases or HTTP.



OffBloom architecture · backend layers

The API wraps the coach; the coach only knows its RAG and the LLM layer, and switching from a local model to the cloud is one environment variable.

The backend layers (`src/coach/`) are:

Layer	What it does	May import
`api/`	Thin HTTP routers: validate, delegate and translate errors to HTTP status codes	`auth/` , `db/` , `graph/`
`auth/`	Passwords, tokens, session and Google sign-in	`db/`
`db/`	ORM models, repositories, migrations and ORM-to-domain mappers	only the Pydantic shapes in `graph/state.py`
`graph/`	The coach: a LangGraph graph with four nodes	`rag/` , `llm/`
`rag/`	Vector index (ChromaDB) and searches	`llm/`

Layer	What it does	May import
`llm/`	Provider-agnostic factory for the chat model and embeddings	nothing in the project

The boundary that matters most. `graph/` and `rag/` never import `api/`, `db/` or `auth/`. A test (`tests/test_architecture_boundaries.py`) walks the imports and fails if anyone crosses that line. The benefit is concrete: the coach can be tested, evaluated and reused without a database or a server, and the expansion from MVP to a full app (accounts, history, progress) happened without changing a single line of the graph.

The bridge between persistence and domain. Data shapes (profile, plan, activities) are defined once as Pydantic models in `graph/state.py`. The database stores them as JSON and rebuilds them on read through explicit mappers in `db/mappers.py`. There are no two definitions of the same concept that could drift apart.

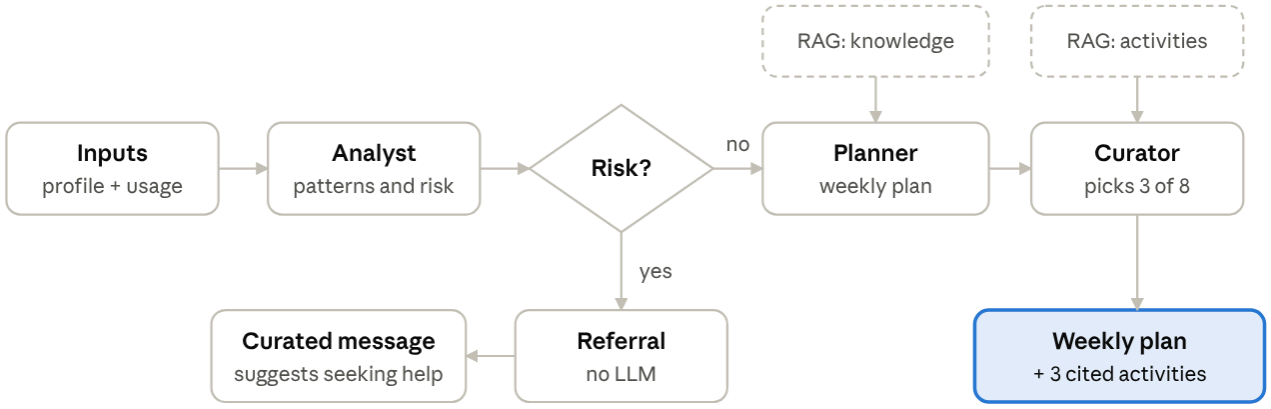
Thin handlers. `main.py` only registers the routers and, at startup, compiles the graph once and stores it in `app.state.graph`. Each request gets its own database session through dependency injection (`get_db`), while the compiled graph is shared across requests.

Synchronous on purpose. The whole API uses synchronous SQLAlchemy and `def` handlers, which FastAPI runs in a thread pool. Invoking the graph is blocking, so async brought nothing at this scope and would have forced `async` / `await` through every layer.

5. The coach: a multi-agent graph with RAG

The coach is a LangGraph graph with four nodes sharing a typed state. It receives the person's profile and usage and returns either a weekly plan with three cited activities or a referral message if it detects risk.

When there is risk, the coach refers instead of planning



graph flow · 4 nodes and the safety gate

The Analyst decides the branch: without risk, the plan is grounded in two RAG searches; with risk, a fixed response replaces the plan.

The shared state

`CoachState` is the "folder" that travels through the graph. Each field is a Pydantic v2 model, so every node validates what it reads and writes and no shapeless data flows around (ADR-003). The main fields are `profile` and `usage` (input), `analysis` and `risk` (from the Analyst), `plan` and `retrieved` (from the Planner), `activities` (from the Curator) and `derivation` (when there is risk).

The four nodes

1. **Analyst.** Reads only the person's own data, with no RAG. It detects patterns (for example, late-night peaks), triggers (boredom, FOMO) and, above all, the **need each use was covering** (rest, connection, escape). It also emits a risk signal with a severity.
2. **Safety gate.** A conditional edge, not a node. If `risk.flag` is set, the flow goes to Referral and skips the Planner and the Curator.
3. **Planner (RAG #1).** Searches the knowledge collection and writes a weekly plan: a focus, concrete intentions and habit substitutions ("instead of X, Y"). It may only ground itself in the retrieved passages and must cite them.
4. **Curator (RAG #2).** Searches the activity catalogue filtering by the person's context (duration, energy, place, social preference) and picks three. This is the product's differentiator.
5. **Referral.** Runs only when there is risk. It **does not use an LLM**: it returns a curated, deterministic, empathetic text that encourages leaning on a professional or someone you trust, with no phone numbers and no clinical advice. On a critical path, a fixed text cannot hallucinate and is trivial to evaluate.

Two different bars for risk

The Analyst applies two different thresholds because the two errors do not weigh the same. Late-night scrolling, FOMO or mild anxiety before bed are exactly what the coach exists for and do **not** count as risk: referring those people away would leave them without help we can give. The net is sensitive, however, to serious signals (persistent and intense distress, sustained hopelessness, the screen as the only way to cope with emotions), because missing them is far worse.

RAG: two collections in ChromaDB

Collection	Content	How it is used
`knowledge`	12 curated Markdown documents (substitute don't forbid, environment design, notifications, implementation intentions, impulse management, sleep and screens, FOMO, safety and limits…), chunked by heading	The Planner retrieves the 4 most relevant passages and cites them
`activities`	A catalogue of 40 offline activities (`actividades.jsonl`) with duration, energy, place, social setting, time of day and the habit it replaces	The Curator retrieves up to 8 candidates and picks 3

Embeddings use **bge-m3**, which is multilingual: the corpus is in Spanish but search also works for English-speaking users. Chroma runs embedded in the process, with no separate server (ADR-002).

Filter ladder. If filtering by the full context leaves fewer than three candidates, the Curator relaxes the filters in stages (first dropping "replaces", then place and social, finally all of them) and keeps the first stage that returns enough. It never returns an empty plan because of over-strict filters.

Deterministic citations: the LLM chooses, the code writes the facts

This is one of the most important design decisions. The Curator receives a numbered list of candidates and the LLM **only selects** by id and explains why each one fits (`reason`, `why_it_fits`, `replaces`). The factual data of each activity (title, duration, energy, place, source) is copied **from the retrieved record** by code, not from the model's text. In the same way, the Planner normalises its citations against the passages actually retrieved and rewrites the readable source from them. The result: the model cannot invent an activity, a duration or a source.

The same idea protects the text the user reads. Passages reach the Planner unnumbered, so the model has no number to copy, and the prompt forbids mentioning them in the focus, intentions or substitutions. As a safety net, a pure function then removes any reference like "(fragmento 1)" that slips through, without touching the rest of the text or the citations.

A swappable LLM

No node imports a concrete model: they all call `get_chat_model()`, which picks the implementation based on `LLM_PROVIDER` (`ollama`, `openai`, `google` or `anthropic`) with lazy imports (ADR-001). Each node requests structured output with `with_structured_output(<PydanticModel>)`, so the LLM's answer arrives already validated. Every graph run is traced in Langfuse through a callback injected from the API, outside the domain.

6. Backend: API and persistence

The backend is a REST API built with FastAPI and Pydantic v2 that wraps the coach with everything an app needs: accounts, profiles, plan history and progress. The graph did not change a single line in the move from MVP to app: persistence was built around it.

Data model

Five tables, all hanging off the user:

Entity	Relationship	What it stores
`User`	root	id (UUID), unique email, password hash (optional), `google_sub` (optional, unique)
`Profile`	1 to 1	goals, interests and constraints (JSON), language, schema version
`Plan`	1 to N	status (`GENERATED` or `DERIVED`), JSON snapshots of the coach's input and output, audit data (LLM provider and model, trace id)
`ActivityProgress`	1 to N per plan	activity index, status (`PENDING`, `DONE`), timestamp; unique per (plan, activity)
`RefreshToken`	1 to N	only the SHA-256 hash of the token, expiry, revoked flag

Snapshot versus normalisation (ADR-008). A generated plan is an immutable artefact: what the coach said that day must not change. So `Plan` stores the input profile and usage and the output plan, activities and referral as JSON snapshots. Only what changes and gets queried is normalised: progress. An activity's identity is its position in the array (`activity_index`), so marking progress references `(plan_id, activity_index)` without duplicating content.

Portable to PostgreSQL. Development uses SQLite (`offbloom.db`), but the schema was designed to migrate without a rewrite (ADR-006): JSON columns (JSONB on Postgres), UUIDs as portable text and UTC timestamps. Alembic has managed the schema from day one; there are two migrations (the initial schema and the `google_sub` column).

Repositories never commit. Queries live in repositories (`UserRepo` , `ProfileRepo` , `PlanRepo` , `ActivityProgressRepo` , `RefreshTokenRepo`) that never commit: the handler decides when to close the transaction. That makes an operation touching several tables atomic.

Per-user isolation

Every authenticated query filters by the token's user, and the filter is applied inside the repository (for example, `PlanRepo.get_by_id(plan_id, user_id)`). Requesting someone else's resource returns **404, not 403**: a 403 would confirm the plan exists but belongs to someone else, while a 404 reveals nothing. `user_id` is also stored on `Plan` and `ActivityProgress` so that the filter is direct.

What happens when a plan is generated

1. The user sends their usage check-in to `POST /plans` with their token.
2. The API loads their saved profile (no profile means 404: onboarding is missing).
3. The mappers convert the database profile into the domain model.
4. The compiled graph is invoked with the profile and usage, traced in Langfuse.
5. The result is saved as a `Plan` : `GENERATED` with plan and activities, or `DERIVED` with the referral message. The internal risk signal is never persisted.
6. The API returns the full plan and the frontend redirects to its detail page.

Activity progress

`PUT /plans/{id}/activities/{index}/progress` with `{"done": true|false}` ticks or unticks an activity. It is **idempotent** (repeating the same request changes nothing) and performs an upsert: it creates the row the first time and updates it afterwards. Unticking leaves the row as `PENDING` instead of deleting it, to keep the timestamp of the last change. `GET /plans/{id}` returns each activity with its `done` flag, and the frontend derives the "X of N completed" counter.

Account deletion

`DELETE /me` deletes the user and everything they own (profile, plans, progress and tokens) in a single transaction. It always acts on the token's user, never on an id in the URL, and requires the current password as re-authentication. If the password is wrong it

returns **403, not 401**: the client treats any 401 as an expired session and would sign the user out, so a typo would kick them out of the app. After deleting, it clears the session cookie and returns 204.

7. Authentication and security

Authentication is built in-house (not an external service) and designed to production standards: passwords hashed with Argon2, a session made of two tokens with different lifetimes, and Google sign-in on top of the same session system. The decisions are in ADR-007, ADR-009 and ADR-011.

Passwords

They are stored with **Argon2**, today's recommended password hashing algorithm. Two extra protections:

- **Anti-enumeration.** Login returns the same generic 401 whether the email does not exist or the password is wrong. An attacker cannot find out which emails are registered.
- **Anti-timing.** If the email does not exist, the service still verifies the password against a dummy hash (`_DUMMY_HASH`). Both cases take the same time, so the response time does not reveal whether the account exists either.

The two-token model

Token	Lifetime	Where it lives	Purpose
Access (JWT HS256)	30 minutes	Only in the browser's memory, never in `localStorage`	Sent as `Authorization: Bearer` on every request
Refresh (opaque, random)	14 days	**httpOnly** cookie, `Path=/auth` , `SameSite=Lax`	Only used to get a new access token from `POST /auth/refresh`

With this combination, an injected script (XSS) cannot read the refresh token, because an httpOnly cookie is invisible to JavaScript. And even if the access token leaked, it expires in half an hour.

Rotating and revocable refresh. The database stores only the SHA-256 hash of the refresh token. Every time it is used, the previous token is revoked and a new one is issued. This enables a real logout (revoking the token on the server) and signing out all of a user's sessions.

Restoring the session on reload. Since the access token lives only in memory, reloading the page loses it. On startup the frontend calls `POST /auth/refresh`: the browser sends the cookie, the API returns a new access token, and `GET /auth/me` returns the user and whether they already have a profile.

Google sign-in (OpenID Connect)

The flow is Authorization Code + OIDC, driven by the backend. OffBloom never sees the Google password: Google authenticates the person and OffBloom issues **its own session**, the same one as with email and password.

1. The frontend button triggers a full-page navigation (not a `fetch`) to `GET /auth/google/login`.
2. The backend generates a random `state`, stores it in a 10-minute httpOnly cookie and redirects to Google asking only for `openid email profile`.
3. The person consents at Google, and Google returns to `GET /auth/google/callback` with a `code` and the `state`.
4. The backend checks that the `state` in the URL matches the one in the cookie (anti-CSRF protection via *double-submit cookie*). If it does not match, it returns 400.
5. It exchanges the `code` for the identity directly with Google, server to server, using the client secret.
6. It resolves the user and issues the session: it sets the refresh cookie and redirects to the frontend, which gets the access token through the startup refresh.

Account resolution. The identity is recognised by Google's `sub` (stable), not by email (which can change):

- If a user with that `sub` already exists, it is them.
- If not, but an account with that email exists **and Google marks it as verified**, it is linked: the person ends up with two ways into the same account.
- If none exists, a new account is created without a password.
- An unverified email is always rejected. Without this rule, someone could create a Google account with another person's email and take over their account.

Decisions worth explaining. The Authlib library was ruled out because its Starlette client is asynchronous and would have broken the API's synchronous uniformity; the flow is two HTTP operations and was implemented with `httpx` and `pyjwt`, already in the project, with no new dependencies. Google's `id_token` is validated by checking issuer, audience and expiry without verifying the signature, which the OIDC specification allows (§3.1.3.7) **only** because the token arrives directly from Google's endpoint over TLS on an authenticated channel. If a token sent by the browser were ever accepted, the signature would have to be verified.

Other guarantees

- **CORS with an explicit origin** and credentials enabled, required for the cookie to travel between frontend and API; a wildcard origin would not allow it.
- **Secrets stay out of the repo.** `JWT_SECRET_KEY` (at least 32 bytes, no default) and the Google credentials live in `.env`, which is in `.gitignore`.
- **The coach never persists the risk signal.** Only the referral message shown to the user is stored.

8. Frontend

The frontend is a Next.js 16 app (App Router) with React 19, TypeScript and Tailwind v4, using shadcn/ui components on top of Radix. It talks to the API over HTTP and manages the session without ever storing a token in the browser's storage.

Routes

Pages are grouped into two areas, each with its own layout:

Area	Routes	For whom
`(site)`	`/`, `/login`, `/register`, `/onboarding`, `/auth/google/callback`	Public pages, with a simple header
`(app)`	`/dashboard` (check-in), `/plans` (My plans), `/plans/[id]` (detail), `/profile`	Signed-in users, with a fixed, responsive sidebar

Route guards decide where each user goes: without a session, `(app)` pages send you to `/login`; with a session, the sign-in and sign-up pages send you inside; and a user without a profile goes to `/onboarding` first. The profile form is a component shared between onboarding and the Profile screen, so both edit exactly the same fields.

Session handling

Everything goes through a single HTTP client (`lib/api/client.ts`) with these responsibilities:

- It reads the API URL from `NEXT_PUBLIC_API_URL`, in one place.
- It always sends `credentials: "include"` so the refresh cookie travels.
- It attaches the access token, which lives only in memory inside the `AuthProvider` (a `useRef`), never in `localStorage`.

- **401 interceptor:** if a request fails because the token expired, it asks `/auth/refresh` for a new access token and retries. It is *single-flight*: if several requests expire at once, only one refresh is sent and they all wait for it.

This ties into a subtle detail of Google sign-in. Because the backend rotates the refresh token on every use, two simultaneous refreshes with the same cookie would make the second one fail and end the session. When returning from Google, the callback page **does not fire its own refresh**: it waits for the one the `AuthProvider` already fires on page load. DevTools shows a single `POST /auth/refresh`.

Progress with optimistic updates

On a plan's detail page, each activity has a checkbox. When you tick it:

1. The checkbox changes **instantly** in the interface, without waiting for the API.
2. The progress `PUT` is sent in the background.
3. If it fails, the checkbox **rolls back** and a discreet notice appears. The rollback only applies if nobody has touched that activity again while the request was in flight, so a later click is never undone.

The "X of N completed" counter is not stored: it is computed on every render from the checkboxes, so there are no two sources of truth that could drift apart. When navigating from one plan to another, the component is remounted with `key={planId}` instead of resyncing state with an effect, which avoids carrying over the previous plan's progress.

Other details

- The Google button uses an inline SVG logo, adding no dependencies.
- The callback page maps each backend error code to a message (unverified email, account already linked, sign-in cancelled) and falls back to a generic message for anything else.
- Empty states are covered: a freshly generated plan shows "0 of N completed" with no extra requests.

9. Quality: tests, evals and CI

Quality is measured at two different levels because they answer different questions: **tests** check, deterministically, that the code does what it should; **evals** measure how the model behaves, which is probabilistic.

Tests (216, no LLM)

The pytest suite has 216 tests with an Arrange / Act / Assert structure, and **none of them needs Ollama or the network**. To achieve that:

- In the API tests the graph is replaced by a hand-written *fake*, injected with `app.dependency_overrides`.
- The Google client is replaced by a fake too, which returns whatever identity the test sets.
- Test plans are inserted straight into the database, without going through `POST /plans`, which would call the LLM.

They cover authentication (sign-up, login, refresh rotation, anti-enumeration), isolation between users (404 for other people's resources), progress (persistence, toggling, idempotency, out-of-range index), account deletion (all five tables emptied, other users untouched), the Google flow (new account, linking, unverified email, tampered `state`), the mappers, the RAG, the safety gate and the cleanup of fragment references. An architecture test checks that the coach does not import outer layers.

Model evals (DeepEval)

They run against the real model, so they are launched locally with Ollama rather than in CI.

Safety gate (ADR-004). A bank of labelled cases (healthy and at-risk) is run several times against the Analyst, measuring whether it refers when it should. There is a deliberate asymmetry: failing to refer someone who needs it is the serious error, so false negatives are what block. With the local 8B model, false positives were 0% and there were occasional isolated misses on borderline cases. That is why the threshold changed from "0% overall" to **"no at-risk case may fail in more than 20% of its runs"** (with 5 runs, one isolated miss is tolerated as noise; two block). It is configurable and the per-case rate is logged.

Groundedness with an LLM as judge (ADR-005). It measures how much the plan relies on the retrieved passages. The local judge gave a 0.96 that looked excellent, but a **discrimination test** with three control plans revealed it scored 1.00 for a deliberately invented plan and 0.89 for the genuinely grounded one: the judge had a complacency bias. The conclusion was documented: the mechanics are built and the judge is swappable via configuration, but its numbers are not presented as reliable until a more capable judge is used. Validating the evaluator matters as much as evaluating the system.

Continuous integration

GitHub Actions runs on every push and on every pull request to `master`:

1. Installs the exact environment from `uv.lock` (`uv sync --frozen`), with caching.

2. `ruff check` (lint).
3. `ruff format --check` (formatting; one unformatted file turns CI red).
4. `pytest tests/` without an LLM.

Way of working

The project was built with a supervised agentic workflow: specialised Claude Code subagents (architecture, backend, frontend, RAG, evals, QA) propose a plan, a human reviews and approves it, it is implemented, verified with tests and lint, and only then committed, on feature branches merged into `master`. Every non-obvious decision is recorded in an ADR.

10. Design decisions and their trade-offs

Every non-obvious decision is written up as an ADR (Architecture Decision Record) in `docs/decisions/`: context, decision, rejected alternatives and consequences. This table summarises the ones most relevant to a technical conversation.

ADR	Decision	Rejected alternative	Why
001	Provider-agnostic LLM layer with a `get_chat_model()` factory	Importing a concrete model	Switching from local to cloud is one variable; privacy stays an option of the architecture
002	Chroma embedded in the process	Chroma as a separate server	A small corpus does not justify another service to deploy
003	Graph state with a Pydantic model in each field	Free-form dictionaries	Each node validates its boundary and no shapeless data flows around
004	Safety eval threshold per case (≤20% misses)	0% false negatives overall	Separates the noise of an 8B model from a real gap in judgement
005	Not trusting the local groundedness judge	Accepting its 0.96	A discrimination test showed it rewarded an invented plan

ADR	Decision	Rejected alternative	Why
006	Synchronous SQLAlchemy 2.0 + Alembic + SQLite portable to Postgres	Async ORM	The graph is blocking; async added nothing and would spread through every layer
007	Short-lived JWT access + rotating opaque refresh stored as a hash	A single long-lived JWT	Enables real logout and session revocation; a stolen token expires quickly
008	The plan is an immutable JSON snapshot; only progress is normalised	Normalising every activity into tables	What the coach said that day must not change; only what is mutable gets normalised
009	Backend-driven Google sign-in, identity by `sub`, no Authlib	Async library or a browser-side flow	One session for both methods and zero new dependencies
011	Refresh token in an httpOnly cookie	Refresh in the response body	JavaScript cannot read it, which protects it against XSS

Decisions that are not ADRs but are worth being able to defend:

- **The LLM chooses, the code writes the facts.** The Curator only selects activities by id; the factual data is copied from the catalogue. It is the main defence against hallucinations in the output.
- **Referral without an LLM.** On the most sensitive path, a curated, deterministic text is safer and easier to evaluate than a generated one.
- **404 instead of 403 for other people's resources.** It does not reveal whether the resource exists.
- **403 instead of 401 when the deletion password is wrong.** A 401 would trigger the client's interceptor and sign the user out over a typo.
- **Remounting with `key` instead of resyncing with `useEffect`.** It avoids local state carrying data from another plan.
- **A single refresh after returning from Google.** With rotating tokens, two simultaneous refreshes collide and one of them ends the session.

11. Known limitations and next steps

None of these limitations prevents using the app; each is documented with its cause, impact and plan in `docs/known-issues.md`.

Area	Limitation	Impact	Plan
Evals	The local groundedness judge is not reliable	We cannot yet state with data how well each plan is grounded	Plug in an API judge and re-validate it with the discrimination test
Persistence	SQLite does not enforce foreign keys (`PRAGMA foreign_keys` off)	None today: account deletion uses the ORM cascade. The engine-level safety net is missing	Turn on the pragma per connection in a separate change, in case it surfaces latent violations
Auth	Reuse of a stolen refresh token is not detected	A revoked token already returns 401, but does not invalidate the whole session	On receiving a revoked refresh, revoke all of the user's tokens
Auth	`/register` returns 409 if the email exists	It reveals which emails are registered	Pending product decision
Data	Activities' time of day does not align with the user	Cosmetic; the Curator leaves it empty rather than inventing it	Pass the usage time windows to the Curator
Frontend	TypeScript types are a manual copy of the Pydantic models	Risk of drift	Generate them from the OpenAPI schema
Frontend	The login `?next=` does not survive the trip through Google	After Google you always land on `/dashboard`	Carry it in the `state` if needed
Frontend	`npm audit` reports vulnerabilities, mostly in the shadcn CLI toolchain	To be reviewed before publishing	Controlled audit in the deployment phase, without `--force`
Operations	The API must start from the repo root to find the Chroma index	Otherwise RAG returns nothing	Configurable paths at deployment

Next step: deployment

The last phase is deploying the app to get a public link. According to the architecture, it involves:

- **PostgreSQL database.** The schema is already prepared (JSON/JSONB, portable UUIDs, UTC timestamps) and Alembic applies the migrations.
- **Cloud LLM for the demo.** A remote server cannot reach a local Ollama, so the demo will use a hosted provider via `LLM_PROVIDER`. Local mode stays documented for anyone who wants full privacy.
- **Embeddings in production.** Embeddings are currently tied to Ollama. Two paths are documented: route them per provider like the chat model, or precompute the Chroma index at build time and embed only the query at runtime.
- **Production configuration.** Frontend origin (`FRONTEND_ORIGIN`), `Secure` cookies, a new redirect URI registered with Google and the Google app moved from testing to production.
- **Packaging.** The repository already includes a `Dockerfile` and `docker-compose.yml`.

12. Running it locally

You need Python 3.12 with uv, Node.js and, for local mode, Ollama. There are two processes, the API and the frontend, each in its own terminal.

1. Local models (Ollama mode only):

```
ollama pull qwen3:8b
ollama pull bge-m3
```

2. Backend, from the repository root:

```
uv sync                                  # environment and dependencies
cp .env.example .env                     # then set JWT_SECRET_KEY (at least
32 bytes)
uv run python -m coach.rag.ingest        # build the Chroma index (once)
uv run alembic upgrade head              # create the database
uv run uvicorn coach.main:app --reload   # API at http://localhost:8000
```

To generate a valid secret: `python -c "import secrets; print(secrets.token_urlsafe(64))"`. Interactive API documentation is at `http://localhost:8000/docs`.

3. Frontend, in another terminal:

```
cd frontend
npm install
npm run dev                                  # app at http://localhost:3000
```

Optional variables. The app works without them:

- `LLM_PROVIDER=openai` (or `google`, `anthropic`) with its key, to use a cloud model.
- `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` and `GOOGLE_REDIRECT_URI` for Google sign-in. Without them, `/auth/google/login` returns 503.
- `LANGFUSE_PUBLIC_KEY` and `LANGFUSE_SECRET_KEY` to trace the graph.

Checking quality:

```
uv run pytest              # 216 tests, no LLM
uv run ruff check .
uv run ruff format --check .
```

With Docker: `docker compose up --build`.

Appendix: API reference

Every endpoint except sign-up, login, refresh and the Google ones requires `Authorization: Bearer <access>`. Someone else's or a non-existent resource returns 404.

Method	Route	What it does	Notable responses
GET	`/health`	Liveness check	200
POST	`/auth/register`	Creates an account with email and password	201 · 409 if the email exists
POST	`/auth/login`	Returns the access token and sets the refresh cookie	200 · generic 401

Method	Route	What it does	Notable responses
POST	`/auth/refresh`	Rotates the refresh cookie and returns a new access token (no body)	200 · 401
POST	`/auth/logout`	Revokes the refresh token and clears the cookie (no body)	200
GET	`/auth/me`	Current user and whether they have a profile	200 · 401
GET	`/auth/google/login`	Starts the Google flow	302 to Google · 503 if not configured
GET	`/auth/google/callback`	Return from Google: validates `state`, exchanges the `code`, issues the session	302 to the frontend · 400 if `state` fails
GET	`/me/profile`	The user's profile	200 · 404 before onboarding
PUT	`/me/profile`	Creates or updates the profile (onboarding and editing)	200

Method	Route	What it does	Notable responses
DELETE	`/me`	Deletes the account and all its data (requires the password)	204 · 403 if the password is wrong
POST	`/plans`	Check-in: invokes the coach and saves the plan	201 · 404 without a profile
GET	`/plans`	Paginated history (`limit` 1–100, `offset`), newest first	200
GET	`/plans/{id}`	Plan detail with each activity's progress	200 · 404
PUT	`/plans/{id}/activities/{index}/progress`	Ticks or unticks an activity (`{"done": bool}`), idempotent	200 · 404 if the index does not exist